UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE FÍSICA
BACHARELADO EM ENGENHARIA FÍSICA

Natália Capra Ferrazzo

UNIGOU Remote Publication:

A Research Paper Submission

Porto Alegre
2022

RESUMO

Por meio do Instituto de Cooperação Acadêmica Tcheco-Brasileira (INCBAC Institute), foi estabelecida a elaboração e desenvolvimento de um artigo propondo uma nova abordagem para o problema matemático NP-hard do Caixeiro Viajante. Utilizando o ambiente IBM Quantum®, o problema foi analisado e descrito e sua solução implementada e documentada.

Palavras-chave: INCBAC; caixeiro-viajante; IBM.

SUMÁRIO

1 INTRODUÇÃO 3

1.1 CONTEXTUALIZAÇÃO 3

1.1.1 Objetivo 3

1.1.2 Diretrizes 4

2 FUNDAMENTAÇÃO TEÓRICA 5

2.1 COMPUTAÇÃO QUÂNTICA 5

2.1.1 Conceitos Básicos 5

2.1.2 Plataforma IBM Quantum® 10

2.2 O PROBLEMA DO CAIXEIRO VIAJANTE 11

2.2.1 Relevância 12

3 METODOLOGIA 13

3.1 CÁLCULO DO PRODUTO TENSORIAL NO QISKIT 21

3.2 CONSTRUINDO O CIRCUITO 25

4 RESULTADOS 26

5 DISCUSSÃO 27

6 CONCLUSÃO 27

7 CONSIDERAÇÕES FINAIS 28

1 INTRODUÇÃO

Através do estudo desenvolvido no Projeto de Elaboração *"UNIGOU Remote Research Practice: A Scientific Training Course"*, foi proposto uma pesquisa acadêmica e a elaboração e desenvolvimento de um artigo sobre o tópico: "*Solving the Travelling Salesman Problem on IBM Quantum®"*.

1.1 CONTEXTUALIZAÇÃO

O Problema do Caixeiro Viajante (*Travelling Salesman Problem*, ou simplesmente TSP) é um problema clássico de otimização no campo da ciência da computação. Pertence a uma classe intrigante de problemas de otimização "difíceis" chamados NP-hard (KUMLANDER, 2008). Neste trabalho, será implementado um algoritmo para resolução do problema na plataforma IBM Quantum® (IBM, 2022).

A plataforma IBM Quantum disponibiliza diversas ferramentas para implementação de algoritmos quânticos. Dentro da plataforma, há a opção de utilizar o IBM Quantum Composer® para implementar circuitos quânticos de uma maneira gráfica. Para esse trabalho, porém, preferiu-se utilizar o IBM Quantum Lab®, que fornece acesso a Jupyter Notebooks onde é possível utilizar o Qiskit, um SDK de código aberto para trabalhar com computadores quânticos no nível de pulsos, circuitos e módulos de aplicativos em linguagem Python.

1.1.1 Objetivo

O objetivo desse trabalho é o desenvolvimento do artigo acadêmico que foi elaborado anteriormente por meio da implementação do problema NP-hard do Caixeiro Viajante dentro das diretrizes estabelecidas pelo programa utilizando a plataforma IBM Quantum Lab®.

Um resultado secundário do projeto é fornecer e apresentar uma implementação de alta qualidade técnica que seja adequada para fins educacionais futuros.

Os trabalhos serão publicados na mídia oficial do INCBAC e a ideia geral é que cada participante do programa selecione um tema relevante de interesse e prepare um trabalho UNIGOU Remoto com base em uma revisão minuciosa da literatura. Em seguida, o artigo resume os achados obtidos durante a revisão da literatura, mencionando as fontes originais. Assim, futuros leitores podem usar seu artigo como inspiração, fonte de informação confiável e base de seu próprio trabalho (INCBAC INSTITUTE, 2022).

1.1.2 Diretrizes

Para o desenvolvimento do artigo, um modelo padrão foi disponibilizado pelo instituto. O *template* fornece o design da publicação requerida pelo programa UNIGOU Remoto para ajudar a compor o artigo de forma satisfatória. É obrigatório o uso do modelo fornecido.

A extensão mínima do artigo é de 4 páginas. A extensão máxima do artigo é de 6 páginas, incluindo as referências.

Devido a ambientação internacional do programa, o trabalho deve ser submetido integralmente na Língua Inglesa.

Todas as imagens, ilustrações e demais gráficos utilizados no trabalho devem ser de boa qualidade e devidamente endereçados aos direitos autorais.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 COMPUTAÇÃO QUÂNTICA

A Mecânica Quântica representa uma parte da Física Moderna que descreve as leis e o comportamento das partículas em escala subatômica. Os principais aspectos do mundo quântico incluem a noção de *superposição*, *emaranhamento*, *interferência* e *natureza probabilística* de processos que podem ser observados nesta área. Esses elementos são amplamente utilizados também na Computação Quântica.

2.1.1 Conceitos Básicos

Para aplicar de maneira satisfatória o algoritmo de resolução do Problema do Caixeiro Viajante, conceitos básicos de Computação Quântica se fazem necessários.

2.1.1.1 Qubits

A computação clássica é baseada em bit – um dígito binário que pode estar em um de dois estados mutuamente exclusivos, 0 ou 1. A computação quântica, por outro lado, é baseada em *qubit* – elemento que pode estar em uma superposição linear complexa de 0 e 1, antes de ser medido. Quando é medido, ele "colapsa" para um desses dois valores, correspondendo aos bits clássicos. Quando um qubit está em uma superposição de estados, pode-se dizer que ele possui uma amplitude associada a cada estado.

Formalmente, um qubit é representado por um vetor unitário, $|\psi\rangle$ de um espaço de Hilbert bidimensional:

$$|\psi\rangle = \alpha|0\rangle + \beta|1$$

Onde $\alpha, \beta \in C$ e normalizados:

$$|\alpha|^2 + |\beta|^2 = 1$$

Desta forma, $|\alpha|^2$ pode ser interpretado como a probabilidade de medir $|0\rangle$ e $|\beta|^2$ como a probabilidade de medir $|1\rangle$. Já $|0\rangle$ e $|1\rangle$ são os estados básicos do espaço de Hilbert e podem ser representados por:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

2.1.1.2 Esfera de Bloch

Existe uma representação alternativa que permite uma melhor visualização de um qubit – a Esfera de Bloch, ilustrada na **Figura 1**. Para projetá-la, é necessário usar a seguinte representação de um qubit, derivada da forma polar dos números complexos:

$$cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi} sin\left(\frac{\theta}{2}\right)|1\rangle$$

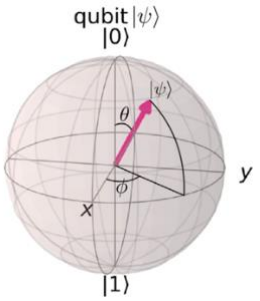


Figura 1. Representação genérica da Esfera de Bloch

2.1.1.3 Portas Quânticas

A evolução de um sistema quântico é descrita por operadores lineares especiais, operadores unitários *U*, que operam em *qubits* da seguinte forma:

$$U|\psi\rangle = \mathrm{U}[\alpha|0\rangle + \beta|1\rangle] = \alpha\mathrm{U}|0\rangle + \beta\mathrm{U}|1\rangle$$

A porta lógica clássica mais comum é o NOT-*gate*, ilustrado na **Figura 2**.

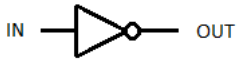


Figura 2. Porta lógica NOT

Seu equivalente quântico é o Pauli-X *gate*, que tem a forma:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Suponha um qubit no estado $|\psi\rangle = 1|0\rangle + 0|1\rangle$. Ao computar$|\psi'\rangle = X|\psi\rangle$, tem-se:

$$|\psi'\rangle = 1 \cdot \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes 0 \cdot \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$|\psi'\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

O resultado mais interessante da porta Pauli-X ocorre quando ela é aplicada a um *qubit* em *estado de superposição*. Nesse caso, o *gate* inverte a probabilidade de que o estado quântico entre em colapso para 0 ou 1.

Para criar um estado de superposição, a porta Hadarmard pode ser aplicada, cujo operador unitário é:

$$H = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}$$

Suponha, novamente, um qubit no estado $|\psi\rangle = 1|0\rangle + 0|1\rangle$. Computando $|\psi'\rangle = H|\psi\rangle$, obtém-se:

$$|\psi'\rangle = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}\left(1 \cdot \begin{bmatrix}1\\0\end{bmatrix} + 0 \cdot \begin{bmatrix}0\\1\end{bmatrix}\right)$$

$$|\psi'\rangle = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\ 1 & -1\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix}$$

$$|\psi'\rangle = \begin{bmatrix}1/\sqrt{2}\\ 1/\sqrt{2}\end{bmatrix}$$

Ou seja, após aplicar o operador quântico H, o qubit estará em estado de *superposição*:

$$|\psi\rangle = \frac{1}{\sqrt{2}} \cdot |0\rangle + \frac{1}{\sqrt{2}} \cdot |1\rangle$$

Neste caso, a probabilidade de que, após medir o qubit, ele seja encontrado no estado $|0\rangle$ ou $|1\rangle$ é a mesma: 50%.

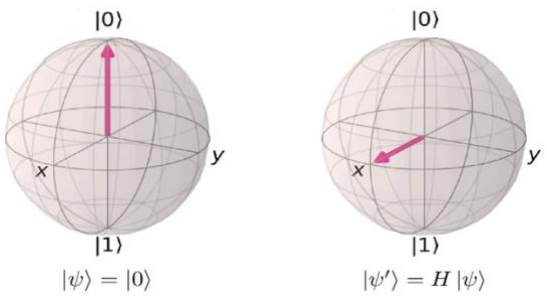


Figura 3. Ilustração da porta lógica Hadamard sendo aplicada a um *qubit* no estado $|0\rangle$: $H|0\rangle$.

Outras portas quânticas interessantes são R_x, R_y e R_z, pois permitem uma modificação simples e direta de magnitude e fase.

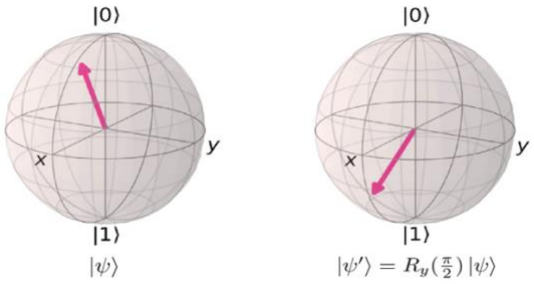


Figura 4. *Gate* $R_y(\frac{\pi}{2})$ aplicado a um *qubit* no estado $|\psi\rangle$

Por exemplo, R_y executa uma rotação em um *qubit* de θ radianos em torno do eixo y. O operador unitário associado ao *gate* R_y é:

$$R_y(\theta) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix}$$

A rotação R_y atua principalmente na magnitude do *qubit*. Assim, esta porta modifica a probabilidade de um *qubit* no estado $|\psi\rangle$ colapsar para 1 ou 0, após a medida. A **Figura 4** mostra um exemplo da aplicação do *gate* $R_y\left(\frac{\pi}{2}\right)$ a um estado quântico genérico $|\psi\rangle$.

2.1.1.4 Sistemas com mais de 1 qubit

Ao lidar com um par de *qubits* tem-se 4 estados possíveis, resultantes do produto tensorial entre ambos:

$$q_0 = |0\rangle,\ q_1 = |0\rangle \rightarrow q = |0\rangle \otimes |0\rangle = |00\rangle$$
$$q_0 = |0\rangle,\ q_1 = |1\rangle \rightarrow q = |0\rangle \otimes |1\rangle = |01\rangle$$
$$q_0 = |1\rangle,\ q_1 = |0\rangle \rightarrow q = |1\rangle \otimes |0\rangle = |10\rangle$$
$$q_0 = |1\rangle,\ q_1 = |1\rangle \rightarrow q = |1\rangle \otimes |1\rangle = |11\rangle$$

E descrever o estado de dois qubits requer quatro amplitudes complexas. Essas amplitudes são armazenadas em um vetor 4D, desse modo:

$$|a\rangle = a_{00}|00\rangle + a_{01}|01\rangle + a_{10}|10\rangle + a_{11}|11\rangle = \begin{bmatrix} a_{00} \\ a_{01} \\ a_{10} \\ a_{11} \end{bmatrix}$$

Um fenômeno muito especial ocorre quando um sistema quântico tem acesso a mais de um *qubit*: o *emaranhamento*.

Pode-se obter um estado emaranhado aplicando uma porta Hadamard seguida de uma porta CNOT, cujo operador é dado por:

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

A porta *CNOT* opera em dois *qubits*, um *qubit* de controle e um *qubit target*. Seu funcionamento é o seguinte: aplica a operação lógica NOT ao *qubit target* somente se o *qubit* de controle tiver o valor 1.

Assim, se uma porta Hadamard é aplicada em q_0 e, após, um CNOT com q_0 como *qubit* de controle e q_1 como *qubit target*, um estado de *emaranhamento* é criado. O circuito quântico correspondente é ilustrado na **Figura 5**.

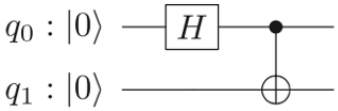


Figura 5. Ilustração do circuito quântica para criação de um estado de *emaranhamento*

2.1.2 Plataforma IBM Quantum®

A IBM fornece algumas ferramentas úteis para computação quântica. Neste relatório, o laboratório IBM Quantum foi usado para implementar todos os algoritmos, e uma introdução das ferramentas é fornecida a seguir.

2.1.2.1 IBM Quantum Composer®

Pode-se facilmente construir circuitos quânticos graficamente com o IBM Quantum Composer®. Ele permite que os usuários vejam dinamicamente algumas propriedades úteis do circuito, como as probabilidades de resultado de medição, vetores de estado, fase e Q-*sphere*. Além disso, fornece a implementação do Qiskit/Python para o circuito atual.

2.1.2.2 IBM Quantum Lab®

Este kit de ferramentas fornece uma coleção de tutoriais Jupyter Notebooks criados pela equipe do Qiskit. Além disso, pode-se criar e rodar seu próprio notebook diretamente neste ambiente. Este ambiente foi amplamente utilizado para a criação deste trabalho.

2.2 O PROBLEMA DO CAIXEIRO VIAJANTE

O Problema do Caixeiro Viajante (*Travelling Salesman Problem*, ou simplesmente TSP) é um problema clássico de otimização no campo da ciência da computação. Pertence a uma classe intrigante de problemas de otimização "difíceis" chamados NP-hard (KUMLANDER, 2008). O problema envolve um vendedor que precisa viajar *N* cidades, visitando cada cidade uma vez e chegando, ao final, à mesma cidade onde começou. Este ciclo de visitar cada cidade uma vez, onde cada cidade representa um único vértice em um grafo, e retornar à cidade inicial é conhecido como ciclo hamiltoniano (BERGE, 1973). Cada cidade está conectada a outras cidades com um custo específico associado a cada conexão. O custo dá uma ideia do quão difícil é atravessar a rota correspondente (que também pode ser interpretado como a distância entre elas). O objetivo do vendedor é minimizar o custo da viagem, satisfazendo as condições acima descritas (SRINIVASAN et al., 2018). Uma ilustração do problema está representada na **Figura 6**.

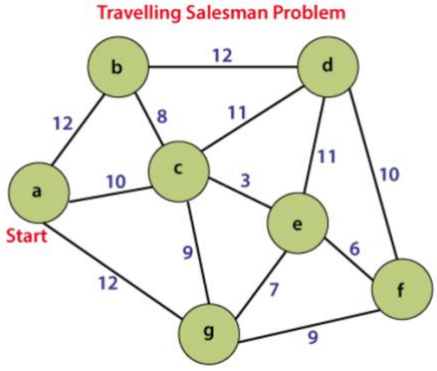


Figura 6. Grafo representando TSP: As letras representam as cidades, e os valores das linhas são as distâncias/custo entre elas.

Mesmo se o problema do caixeiro viajante for dividido em componentes menores, cada componente será pelo menos tão complexa quanto o problema inicial. É por isso que ele pertence à classe de problemas NP-hard. A solução clássica mais cara e simples para o problema é pelo método da força bruta. No entanto, o problema torna-se impossível de resolver quando envolve um grande número de cidades. Para *N* cidades, *(N-1)!* possíveis iterações são necessárias para procurar a solução, que cresce muito rápido à medida que *N* aumenta (SRINIVASAN et al., 2018).

2.2.1 Relevância

O problema do caixeiro viajante (TSP) é muito importante na ciência da computação teórica. A comunidade vem tentando encontrar um algoritmo que possa resolvê-lo em tempo polinomial. A razão de sua importância não é especificamente o TSP, mas sim a classe à qual o TSP pertence, a classe *NP*. Se for possível encontrar um algoritmo para resolver o TSP, ele abrirá uma ampla gama de possibilidades para os milhares de outros problemas que pertencem à mesma classe (ANIS et al., 2021).

3 METODOLOGIA

Este trabalho propõe um algoritmo quântico para resolver o TSP usando a técnica de *estimativa de fase*. O problema é abordado codificando os custos entre as cidades como fases. Construiu-se operadores unitários cujos *auto-vetores* são os estados da base computacional e os *auto-valores* são combinações dessas fases. Em seguida, aplicou-se o algoritmo de estimativa de fase para os auto-estados que dão todos os custos totais possíveis para todas as rotas. Depois de obter os custos, é possível pesquisar essas informações usando o "*Algoritmo de busca quântica para encontrar o mínimo*" (DÜRR; HØYER, 1999) para encontrar o menor custo possível, bem como a rota percorrida. Esta forma de abordagem fornece uma aceleração quadrática em comparação ao método clássico de força bruta para um grande número de cidades. Neste artigo, foi ilustrado um exemplo TSP com quatro cidades.

Para começar, considera-se todo o problema em termos de grafos. As *n* cidades são representadas por vértices/nós e o custo entre elas como arestas. Para o caso desse estudo, $n = 4$. A **Figura 7** ilustra o problema através de um grafo. Nela, as cidades são representadas por círculos enumerados de 1 a 4 e os custos de cada caminho que liga as cidades são rotuladas nelas, onde, $\phi_{i \to j}$ representa o custo da cidade *i* para a cidade *j*. Este é o caso mais geral do TSP, onde o custo para viajar da cidade *i* para a cidade *j* não é o mesmo que o custo para viajar da cidade *j* para a cidade *i*.

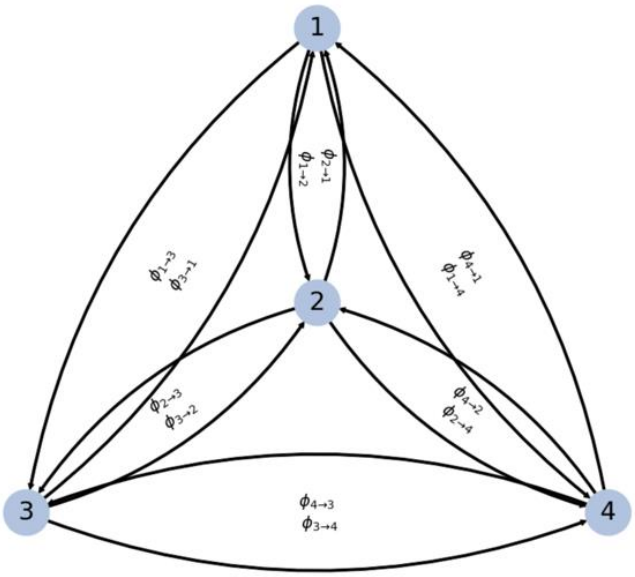


Figura 7. Ilustração de grafo com $n = 4$

O método clássico de força bruta utiliza uma matriz $n \times n$ para armazenar o custo entre cada par de nós, como representado na matriz ***A***.

$$A = \begin{bmatrix} \phi_{1\to1} & \phi_{1\to2} & \phi_{1\to3} & \phi_{1\to4} \\ \phi_{2\to1} & \phi_{2\to2} & \phi_{2\to3} & \phi_{2\to4} \\ \phi_{3\to1} & \phi_{3\to2} & \phi_{3\to3} & \phi_{3\to4} \\ \phi_{4\to1} & \phi_{4\to2} & \phi_{4\to3} & \phi_{4\to4} \end{bmatrix}$$

Como o custo de estar em uma cidade e ir para ela mesma é *0*, os elementos da diagonal da matriz *A* são todos *0*. Com isso, dois problemas surgem:

1. A matriz não é unitária, o que implica que a implementação e manipulação dela como operador não é possível em um computador quântico.
2. As fases serão somadas quando as multiplicarmos ou tomarmos produtos tensoriais de estados com essas fases como coeficientes, ou seja, os custos serão somados como fases necessárias para a busca, sendo assim, não podem ser nulos.

Por essa razão, a representação da matriz deve ser ajustada de forma que $B_{ij} = e^{i(\phi_{ij})}$.

$$B = \begin{bmatrix} e^{i\phi_{1\to1}} & e^{i\phi_{1\to2}} & e^{i\phi_{1\to3}} & e^{i\phi_{1\to4}} \\ e^{i\phi_{2\to1}} & e^{i\phi_{2\to2}} & e^{i\phi_{2\to3}} & e^{i\phi_{2\to4}} \\ e^{i\phi_{3\to1}} & e^{i\phi_{3\to2}} & e^{i\phi_{3\to3}} & e^{i\phi_{3\to4}} \\ e^{i\phi_{4\to1}} & e^{i\phi_{4\to2}} & e^{i\phi_{4\to3}} & e^{i\phi_{4\to4}} \end{bmatrix}$$

Dessa forma, todos os elementos da diagonal da matriz *B* são *1*.

O próximo passo é construir uma matriz unitária *U* a partir da matriz *B* para cada cidade/nó com a estimativa de fase como:

$$U_j = \sum_{i=1}^{n} B[j][i] \times \ produto\ externo\ de\ todos\ vetores\ de\ base\ possíveis$$

Onde $j, i \geq 0$ e $j, i \in [1, n]$. O restante dos elementos da matriz U_j são *0*. Basicamente, U_j são matrizes diagonais unitárias construídas a partir de cada uma das colunas da matriz *B*. Por exemplo, U_1 é calculada da seguinte maneira:

$$U_1\colon\ e^{i\phi_{1\to1}}|00\rangle\langle00| + e^{i\phi_{2\to1}}|01\rangle\langle01| + e^{i\phi_{3\to1}}|10\rangle\langle10| + e^{i\phi_{4\to1}}|11\rangle\langle11|$$

No geral, essas matrizes serão $n \times n$. Como nesse projeto $n = 4$, aqui essas matrizes serão 4×4. E serão expressas em termos de vetores de base com as fases como coeficientes.

Uma análise mais profunda de U_1 mostra que a matriz é realmente uma matriz diagonal unitária, onde $|\alpha\alpha\rangle\langle\alpha\alpha|$ expressa a posição do coeficiente não nulo da matriz:

$$|00\rangle\langle00| = \left(\begin{bmatrix}1\\0\end{bmatrix} \otimes \begin{bmatrix}1\\0\end{bmatrix}\right) \otimes \left(\begin{bmatrix}1 & 0\end{bmatrix} \otimes \begin{bmatrix}1 & 0\end{bmatrix}\right) = \begin{bmatrix}1\\0\\0\\0\end{bmatrix} \otimes \begin{bmatrix}1 & 0 & 0 & 0\end{bmatrix}$$

$$|00\rangle\langle 00| = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

De maneira análoga, ao calcular os demais elementos, conclui-se que:

$$|01\rangle\langle 01| = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$|10\rangle\langle 10| = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$|11\rangle\langle 11| = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Fica evidente que, ao multiplicar essas matrizes pelos coeficientes de fases e somá-las, resulta em:

$$U_1 = \begin{bmatrix} e^{i\phi_{1\to 1}} & 0 & 0 & 0 \\ 0 & e^{i\phi_{2\to 1}} & 0 & 0 \\ 0 & 0 & e^{i\phi_{3\to 1}} & 0 \\ 0 & 0 & 0 & e^{i\phi_{4\to 1}} \end{bmatrix}$$

Analogamente, segue que:

$$U_2 = \begin{bmatrix} e^{i\phi_{1\to 2}} & 0 & 0 & 0 \\ 0 & e^{i\phi_{2\to 2}} & 0 & 0 \\ 0 & 0 & e^{i\phi_{3\to 2}} & 0 \\ 0 & 0 & 0 & e^{i\phi_{4\to 2}} \end{bmatrix}$$

$$U_3 = \begin{bmatrix} e^{i\phi_{1\to3}} & 0 & 0 & 0 \\ 0 & e^{i\phi_{2\to3}} & 0 & 0 \\ 0 & 0 & e^{i\phi_{3\to3}} & 0 \\ 0 & 0 & 0 & e^{i\phi_{4\to3}} \end{bmatrix}$$

$$U_4 = \begin{bmatrix} e^{i\phi_{1\to4}} & 0 & 0 & 0 \\ 0 & e^{i\phi_{2\to4}} & 0 & 0 \\ 0 & 0 & e^{i\phi_{3\to4}} & 0 \\ 0 & 0 & 0 & e^{i\phi_{4\to4}} \end{bmatrix}$$

Como todas as matrizes seguem o mesmo padrão, pode-se generalizar para:

$$U_j = \begin{bmatrix} e^{ia} & 0 & 0 & 0 \\ 0 & e^{ib} & 0 & 0 \\ 0 & 0 & e^{\,ic} & 0 \\ 0 & 0 & 0 & e^{id} \end{bmatrix}$$

De forma que, escolhendo diferente fases *a*, *b*, *c* e *d* retorna U_1, U_2 , U_3 e U_4.

O algoritmo de estimativa de fase quântica usa o *retrocesso de fase* para escrever a fase de U para *t qubits*. Ao utilizar um *qubit* de controle no operador *U*, o *qubit* irá mudar de fase, devido ao retrocesso, em $e^{2i\pi\theta}$. Por isso, é necessária a introdução do controlador unitário *C-U* que aplica o operador unitário *U* no *qubit target* apenas se seu *qubit* de controle é $|1\rangle$. Para isso, U_1, U_2, U_3 e U_4 devem ser convertidos para CU_1, CU_2, CU_3 e CU_4, conforme segue:

$$U_j = \begin{bmatrix} e^{ia} & 0 & 0 & 0 \\ 0 & e^{ib} & 0 & 0 \\ 0 & 0 & e^{\,ic} & 0 \\ 0 & 0 & 0 & e^{id} \end{bmatrix}$$

$$U_j = \left(\begin{bmatrix} 1 & 0 \\ 0 & e^{i(c-a)} \end{bmatrix} \otimes \begin{bmatrix} e^{ia} & 0 \\ 0 & e^{ib} \end{bmatrix} \right) \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i(d-c+a-b)} \end{bmatrix}$$

$$CU \equiv C\,(U1 \otimes U2 \otimes U3 \otimes U4) \equiv CU1 \otimes CU2 \otimes CU3 \otimes CU4$$

Os autovalores da matriz unitária *U* são estimados utilizando o algoritmo de estimativa de fase quântica. As fases podem ser facilmente normalizadas para serem limitadas dentro de 0 e 2π uma vez que a gama de distâncias entre as cidades é conhecida. Assim, *U* é uma matriz diagonal proveniente do produto tensorial entre *n* matrizes diagonais. Isso significa que os autovalores da matriz *U* são os estados de base computacional com autovalores como os elementos diagonais correspondentes.

U é uma matriz diagonal, já que deriva do produto tensorial de matrizes diagonais. Assim, dos n^n elementos da matriz, apenas $(n-1)$ contém comprimento de todos os ciclos hamiltonianos distintos possíveis no TSP. Cada autovetor é representado como um ciclo hamiltoniano distinto. Então, existem $(n-1)!$ autoestados de *U* com autovalores sendo o custo total do ciclo hamiltoniano correspondente como fase. A fase correspondente à rota é estimada passando pelas cidades em alguma ordem genérica e chegando ao estado inicial. Os autoestados são representados em forma binária para converter a cidade em vetores de base computacional usando uma função:

$$|\psi\rangle = \otimes_j |i(j) - 1\rangle$$

Onde $j \in [1 \ldots n]$ e a função $i(j)$ representa de que cidade se viajou para a cidade *j*.

Para 4 cidades, a combinação de todos os ciclos Hamiltonianos distintos é $(n-1)! = (4-1)! = 3! = 6$, ou seja, o número de autovalores é 6. Para entender o que ciclos Hamiltonianos *distintos* significa nesse problema, segue:

Pegando, por exemplo, os caminhos

$$1 - 2 - 3 - 4,$$
$$2 - 3 - 4 - 1,$$
$$3 - 4 - 1 - 2,$$
$$4 - 1 - 2 - 3\,,$$

é possível observar que correspondem a caminhos equivalentes, a menos de uma *permutação circular*. Com isso, tem-se os 6 estados distintos conforme **Tabela 1**.

Estados Possíveis
$\mathbf{1 - 2 - 3 - 4}$
$\mathbf{1 - 2 - 4 - 3}$
$\mathbf{1 - 4 - 2 - 4}$
$\mathbf{1 - 4 - 3 - 2}$
$\mathbf{1 - 3 - 2 - 4}$
$\mathbf{1 - 3 - 4 - 2}$

Tabela 1. Estados distintos

Evidentemente, esse conjunto de 6 estados distintos não é único devido a estados equivalentes derivados de permutação circular sinalizada anteriormente.

Para o caso de o custo de viajar da cidade *i* para a cidade *j* for o mesmo, o número de ciclos Hamiltonianos distintos é $\frac{(n-1)!}{2}$ (BARENCO et al., 1995). Isso se deve ao fato de caminhos como $1 - 2 - 3 - 4$ e $1 - 2 - 4 - 3$ resultarem no mesmo custo total.

Agora, aplicando-se:

$$|\psi\rangle = \otimes_j |i(j) - 1\rangle$$

Em $1 - 2 - 3 - 4$, onde:

$$i(1) = 4$$

Significa que da cidade 4 foi viajado até a cidade 1, converte-se para a forma binária para os estados da base computacional, conforme segue:

$$|i(1) - 1\rangle = |4 - 1\rangle = |3_{10}\rangle = |11_2\rangle$$

Analogamente,

$$|i(2) - 1\rangle = |1 - 1\rangle = |0_{10}\rangle = |00_2\rangle$$
$$|i(3) - 1\rangle = |2 - 1\rangle = |1_{10}\rangle = |01_2\rangle$$
$$|i(4) - 1\rangle = |3 - 1\rangle = |2_{10}\rangle = |10_2\rangle$$

Aplicando o produto tensorial em todos os estados, tem-se os autoestados na forma binária:

$$|00\rangle \otimes |01\rangle \otimes |10\rangle \otimes |11\rangle \ = \ |11000110\rangle$$

Fazendo essa operação para todos os estados distintos da **Tabela 1**, tem-se os seguintes autoestados:

Trajetos	**Autoestados**
$\mathbf{1-2-3-4}$	$\|11000110\rangle$
$\mathbf{1-2-4-3}$	$\|10000111\rangle$
$\mathbf{1-4-2-4}$	$\|10001101\rangle$
$\mathbf{1-4-3-2}$	$\|01001110\rangle$
$\mathbf{1-3-2-4}$	$\|11001001\rangle$
$\mathbf{1-3-4-2}$	$\|01001011\rangle$

Tabela 2. Autoestados e sua forma binária

Para o caso especial em que o custo de viagem de *i* para *j* é o mesmo que de *j* para *i*, tem-se:

Trajetos	**Autoestados**
$\mathbf{1-2-3-4}$	$\|11000110\rangle$
$\mathbf{1-4-2-4}$	$\|10001101\rangle$
$\mathbf{1-3-2-4}$	$\|11001001\rangle$

Tabela 3. Autoestados e sua forma binária para o caso especial.

3.1 CÁLCULO DO PRODUTO TENSORIAL NO QISKIT

Agora que toda teoria foi discutida, é feita a construção do algoritmo parte por parte.

Partindo-se da generalização de U_j e sua decomposição em termos de operadores unitários de controle:

$$U_j = \left(\begin{bmatrix} 1 & 0 \\ 0 & e^{i(c-a)} \end{bmatrix} \otimes \begin{bmatrix} e^{ia} & 0 \\ 0 & e^{ib} \end{bmatrix}\right) \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i(d-c+a-b)} \end{bmatrix}$$

Nessa expressão,

$$\begin{bmatrix} 1 & 0 \\ 0 & e^{i(c-a)} \end{bmatrix}$$

representa um *gate* unitário $U_1(c-a)$. Analogamente,

$$\begin{bmatrix} e^{ia} & 0 \\ 0 & e^{ib} \end{bmatrix}$$

representa um *gate* unitário $U_1(b-a)$, ao fatorar a fase global e^{ia} e fazer um *gate* unitário U_a. Sendo assim, a matriz

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i(d-c+a-b)} \end{bmatrix}$$

é um *gate* unitário $CU_1(d-c+a-b)$. Porém, o problema requer CU_j. Para isso, a matriz U deve mudar de uma matriz 4×4 para uma matriz 8×8, conforme segue:

$$U_j \rightarrow CU_j = \begin{bmatrix} e^{ia} & 0 & 0 & 0 \\ 0 & e^{ib} & 0 & 0 \\ 0 & 0 & e^{ic} & 0 \\ 0 & 0 & 0 & e^{id} \end{bmatrix}$$

$$\rightarrow \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & e^{ia} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & e^{ib} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & e^{ic} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & e^{id} \end{bmatrix}$$

Ou seja, ao converter U_j para $C - U_j$, deve-se controlar todos os elementos em U_j. Então, $U_1(c-a)$ será $CU_1(c-a)$, $U_1(b-a)$ será $CU_1(b-a)$, $U_1(a)$ atua como uma fase global no *qubit target*. E, mais importante, $CU_1(d-c+a-b)$ será *controle-controle de* $U_1(d-c+a-b)$, ou seja, $CCU_1(d-c+a-b)$.

É possível decompor qualquer *gate* unitário *controle-controle* em uma série de *gates* unitários, como discutido em Lemma 6.1, (BARENCO et al., 1995) e ilustrado na **Figura 8**, onde *V* é um operador unitário.

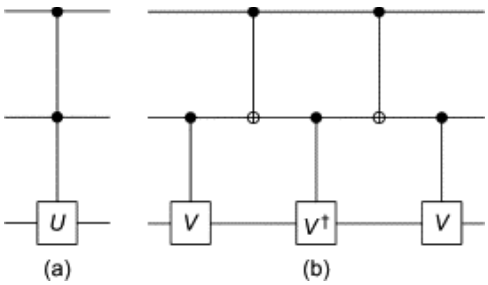


Figura 8. Ilustração da equivalência de *CCU* e série de *gates* unitários *V*.

Utilizando isso, podemos decompor $CCU_1(d-c+a-b)$. Na **Figura 8**, U seria $U_1(d-c+a-b)$, V seria $U_1(d-c+a-b)$ e $V^\dagger$ seria $U_1(-(d-c+a-b))$.

Na **Figura 9** é ilustrado um circuito-teste para análise de $CCU_1(d - c + a - b)$, onde

$$a = 0$$
$$b = {}^{\pi}/_{2}$$
$$c = {}^{\pi}/_{8}$$
$$d = {}^{\pi}/_{4}$$

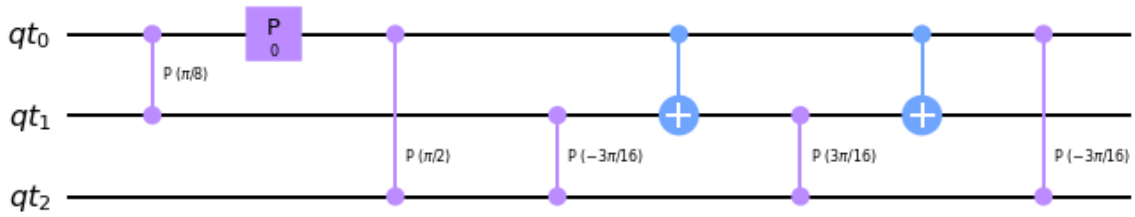


Figura 9. Ilustração da implementação de um circuito-teste para análise de $CCU_1(d - c + a - b)$.

A partir disso, pode-se utilizar a biblioteca Aer para executar o circuito e obter a seguinte matriz unitária:

$$\text{Circuit} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & i & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.92388 + 0.38268i & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}}(1+i) \end{bmatrix}$$

Pode-se notar que a matriz obtida não é exatamente igual à esperada:

$$CU_j = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & e^{ia} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & e^{ib} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & e^{ic} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & e^{id} \end{bmatrix}$$

Mas sim uma versão rearranjada dela. Porém, não há nenhum erro: isso ocorre porque o Qiskit interpreta o produto tensorial do *qubit* inferior ao *qubit* superior(QISKIT, 2022). Então, para obter exatamente a mesma forma que o CU_j, seria necessário inverter o circuito:

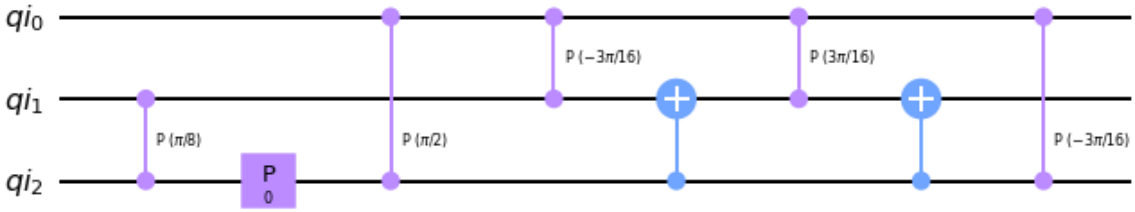


Figura 10. Ilustração da implementação do circuito-teste invertido para análise de $CCU_1(d - c + a - b)$.

E assim, tem-se:

$$\text{Circuit} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & i & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.92388 + 0.38268i & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}}(1+i) \end{bmatrix}$$

Que é totalmente equivalente a U_j.

3.2 CONSTRUINDO O CIRCUITO

Para a construção do circuito, foram consideradas todas as fases conforme **Tabela 4**.

Fase	Representação	Valor
$\boldsymbol{\phi_{1\to1}}$		0
$\boldsymbol{\phi_{1\to2}}$	*a*	$\pi/2$
$\boldsymbol{\phi_{1\to3}}$	*b*	$\pi/8$
$\boldsymbol{\phi_{1\to4}}$	*c*	$\pi/4$
------	------	------
$\boldsymbol{\phi_{2\to1}}$	*d*	$\pi/2$
$\boldsymbol{\phi_{2\to2}}$		0
$\boldsymbol{\phi_{2\to3}}$	*e*	$\pi/4$
$\boldsymbol{\phi_{2\to4}}$	*f*	$\pi/4$
------	------	------
$\boldsymbol{\phi_{3\to1}}$	*g*	$\pi/8$
$\boldsymbol{\phi_{3\to2}}$	*h*	$\pi/4$
$\boldsymbol{\phi_{3\to3}}$		0
$\boldsymbol{\phi_{3\to4}}$	*i*	$\pi/8$
------	------	------
$\boldsymbol{\phi_{4\to1}}$	*j*	$\pi/4$
$\boldsymbol{\phi_{4\to2}}$	*k*	$\pi/4$
$\boldsymbol{\phi_{4\to3}}$	*l*	$\pi/8$
$\boldsymbol{\phi_{4\to4}}$		0

Tabela 4. Fases utilizadas na implementação do circuito, com as respectivas representações utilizadas no algoritmo e valores.

A construção final do circuito pode ser visualizada na **Figura 11**.

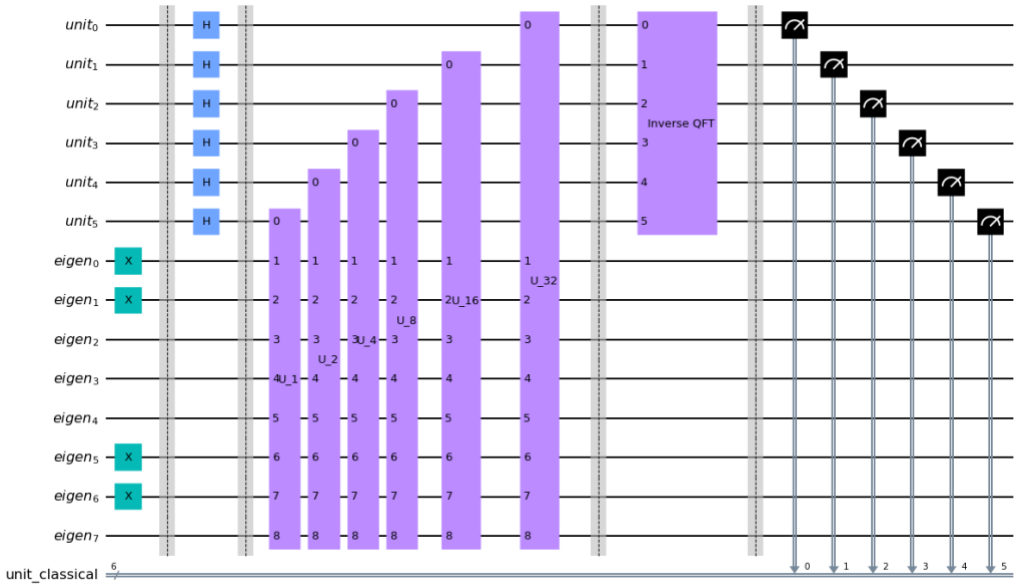


Figura 11. Ilustração do circuito implementado, com *unit* representando os *qubits*, e *eigen* representando os autoestados.

4 RESULTADOS

O resultado obtido pelo algoritmo para o primeiro autoestado está ilustrado na **Figura 12**, que corresponde aos trajetos $\mathbf{1 - 2 - 3 - 4}$ e $\mathbf{1 - 2 - 4 - 3}$.

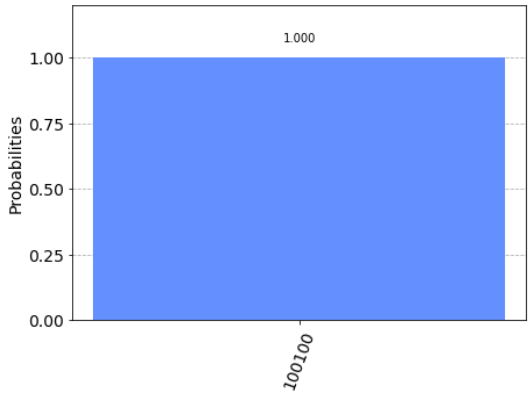


Figura 12. Estado obtido.

5 DISCUSSÃO

O uso desse processo de estimativa de fase quântica para resolver um problema de NP-Hard não fornece um algoritmo mais eficiente e ideal para resolver esses tipos de problemas do que os já disponíveis. Esse processo de usar a estimativa de fases, que codifica o custo como fases, é uma das muitas maneiras de resolver essa categoria de problemas. Nesse processo, foram encontrados todos os ciclos hamiltonianos possíveis no grafo e, com base nesses ciclos, foi calculado o custo total. Mas encontrar todo o ciclo hamiltoniano de um grafo é em si um problema NP-Completo. Foi usado o ciclo hamiltoniano pré-computado no início para obter os autoestados. E apenas 4 nós/cidades foram usados, mas para um grande número de cidades, encontrar todos os ciclos hamiltonianos possíveis é um trabalho extenso.

6 CONCLUSÃO

Apesar de não trazer, *a priori*, nenhum avanço a cerca de resoluções de problemas NP-hard, utilizar um algoritmo de computação quântica para a resolução do Problema do Caixeiro Viajante é uma nova abordagem. Utilizando o algoritmo proposto, foi possível criar um banco de dados de todas as rotas possíveis que podem ser percorridas, juntamente com a distância de cada uma. Isso dá flexibilidade a esse algoritmo, que pode ser explorado no futuro para resolver o problema do caixeiro viajante com muito mais eficiência (SRINIVASAN et al., 2018). Embora este algoritmo trate de um caso muito geral, existem alguns casos que não podem ser resolvidos diretamente com ele. Esses são os casos em que há restrições nas rotas que ligam as cidades. Isso pode ser pensado como a distância entre essas cidades sendo infinita. Como esse algoritmo requer distâncias que podem ser normalizadas de modo que a distância total para a rota mais longa seja menor que 2π, não é um caso que pode ser resolvido utilizando-o.

7 CONSIDERAÇÕES FINAIS

O artigo *"Solving the Travelling Salesman Problem on IBM Quantum®"* desenvolvido nesse trabalho e submetido ao programa UNIGOU se encontra no **Apêndice A**.

Todos os algoritmos/notebooks utilizados nesse estudo estão disponíveis no repositório GitHub[1] da autora. Contudo, também podem ser consultados no **Apêndice B**.

[1] Disponível em: https://github.com/nataliaferrazzo/Solving-the-Travelling-Salesman-Problem-on-IBM-Quantum

REFERÊNCIAS

ANIS, M. D. S. et al. **Qiskit: An Open-source Framework for Quantum Computing**, 2021.

BARENCO, A. et al. **Elementary gates for quantum computation**. [s.l: s.n.].

BERGE, C. **Graphs and Hypergraphs**. [s.l: s.n.]. Disponível em: <http://www.gbv.de/dms/hebis-darmstadt/toc/10727930.pdf>. Acesso em: 24 abr. 2022.

DÜRR, C.; HØYER, P. **A quantum algorithm for finding the minimum**. [s.l: s.n.].

IBM. **IBM Quantum**. Disponível em: <https://quantum-computing.ibm.com/>. Acesso em: 23 abr. 2022.

INCBAC INSTITUTE. **UNIGOU Remote Edital**. [s.l: s.n.]. Disponível em: <https://www.incbac.org/UNIGOU_Remote_Edital.pdf>. Acesso em: 22 abr. 2022.

KUMLANDER, D. **NP-Hard Graph Problems' Algorithms Testing Guidelines: Artificial Intelligence Principles and Testing as a Service**. (M. Iskander, Ed.)Innovative Techniques in Instruction Technology, E-learning, E-assessment, and Education. **Anais**...Dordrecht: Springer Netherlands, 2008.

QISKIT. **Quantum Fourier Transform**. Disponível em: <https://qiskit.org/textbook/ch-algorithms/quantum-fourier-transform.html>. Acesso em: 27 abr. 2022.

SRINIVASAN, K. et al. **Efficient quantum algorithm for solving travelling salesman problem: An IBM quantum experience**. [s.l: s.n.].

I acknowledge the use of IBM Quantum services for this work. The views expressed are those of the author, and do not reflect the official policy or position of IBM or the IBM Quantum team.

Apêndice A – Artigo submetido ao programa UNIGOU

INCBAC INSTITUTE
Institute of Czech-Brazilian Academic Cooperation

UNIGOU Remote 2022
Czech-Brazilian Academic Program

Solving the Travelling Salesman Problem on IBM Quantum®

Natália Capra Ferrazzo

Institute of Physics, Federal University of Rio Grande do Sul, Porto Alegre, Brazil, natalia.ferrazzo@ufrgs.br.

Abstract. In this article, the basic concepts of Quantum Computing were studied and applied to NP-hard Travelling Salesman problem. Through the tools provided by IBM Quantum and Qiskit initiative, it was able to implement it via Python libraries and Jupyter Notebook. The results were as computed for similar publications.

Keywords: NP-hard, Quantum computing, Travelling Salesman Problem

1. Introduction

As Richard Feynman highlighted, some quantum mechanics effects, as superposition and entanglement, can not be simulated efficiently on classical computers [1], raising the conjecture that computation can be done more efficiently if quantum effects are used to perform it. Specifically, as mentioned by Acompora et al. [2], Feynman speculated that the synergistic usage of quantum superposition and entanglement in computation may enable the design of computing devices showing a high degree of parallelism, which grows with the size of the device itself, in an exponential way.

In this way, quantum computers are being used to approach these kinds of problems using various techniques, even though there are supercomputers all over the world. It's because of the fact of what a quantum computer can do over a classical computer, using the weirdness of quantum mechanics and all the different properties which only a quantum computer can provide, and using algorithms developed for quantum computer over the years.

Although the use of quantum computers doesn't guarantee the problem will be solved, it still offers a new way to approach problems of this class.

2. Basic Concepts

2.1 Qubits

Classical computing is based on *bit*: a binary digit that can be in one of two mutual exclusive states, either 0 or 1. Quantum computing, otherwise, is based on *qubit*: it can be in a quantum state that is a complex linear superposition of 0 and 1, before being measured. When it is measured, it *"collapses"* to one of these two values, corresponding to classical bits. When a qubit is in a super-position of states, it can be said that it has an *amplitude* associated with each state.

Formally, a qubit is represented by a unit vector, $|\psi\rangle$ of a two-dimensional Hilbert space:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Where $\alpha, \beta \in \mathbb{C}$ and it's normalized so:

$$|\alpha|^2 + |\beta|^2 = 1$$

In this way, $|\alpha|^2$ can be interpreted as the *probability of measuring* $|0\rangle$ and $|\beta|^2$ as the *probability of measuring* $|1\rangle$.

The $|0\rangle$ and $|1\rangle$ are the basis states of the Hilbert space and can be represented by:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

2.2 Bloch Sphere

There is an alternative representation that allows a better visualization of a qubit – the so-called *Bloch Sphere*. In order to design it, it's needed to use the following representation of a qubit, derived from the polar form of complex numbers:

$$cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi} sin\left(\frac{\theta}{2}\right)|1\rangle$$

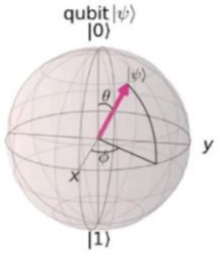


Fig 1. Bloch sphere

2.3 Quantum gates

The evolution of a closed quantum system is described by special linear operators, *unitary operators* U which operate on qubits as follows:

$$U|\psi\rangle = U[\alpha|0\rangle + \beta|1\rangle] = \alpha U|0\rangle + \beta U|1\rangle$$

The most common gate is the NOT-gate, or, in quantum equivalence Pauli-X gate. It has the form of:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Suppose a qubit in state $|\psi\rangle = 1 \cdot |0\rangle + 0 \cdot |1\rangle$. If one compute $|\psi'\rangle = X|\psi\rangle$, gets:

$$|\psi'\rangle = 1 \cdot \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes 0 \cdot \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$|\psi'\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

The most interesting outcome of Pauli-X gate occurs when it is applied to a qubit in superposition state. In such case, it inverts the probability that the quantum state will collapse to 0 or 1.

To create a superposition state, a Hadarmard gate can be applied, the unitary operator is:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Suppose, again, a qubit in state $|\psi\rangle = 1 \cdot |0\rangle + 0 \cdot |1\rangle$. If one compute $|\psi'\rangle = H|\psi\rangle$, gets:

$$|\psi'\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \left(1 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix}\right)$$

$$|\psi'\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$|\psi'\rangle = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}$$

As one can see, after applying the quantum operator H, the qubit will be in a superposition state:

$$|\psi\rangle = \frac{1}{\sqrt{2}} \cdot |0\rangle + \frac{1}{\sqrt{2}} \cdot |1\rangle$$

In this case, the probability that after measuring the qubit, it will be found in state $|0\rangle$ or $|1\rangle$ is the same: 50%.

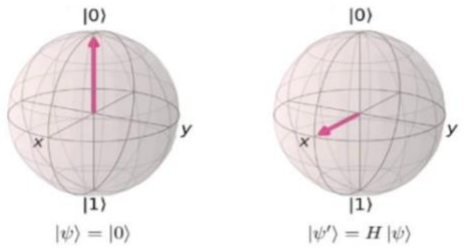


Fig 2. Hadamard gate applied to a qubit in state $|0\rangle$: $H|0\rangle$

Other interesting quantum gates are R_x, R_y and R_z, because they allow a simple and direct modification of the magnitude and phase.

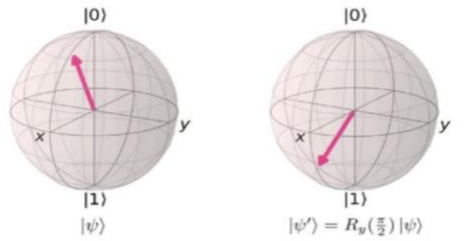


Fig 3. R_y gate applied to a qubit in state $|\psi\rangle$.

For example, R_y performs a single-qubit rotation through angle θ radians around the y-axis. The unitary operator associated with the R_y gate is:

$$R_y(\theta) = \begin{bmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{bmatrix}$$

The R_y rotation mainly acts on the magnitude knob of qubit. Thus, this gate modifies the probability that a qubit in state $|\psi\rangle$ will collapse to 1 or 0, after measuring it. **Fig 3** shows an example of the application of the gate $R_y(\frac{\pi}{2})$ to a generic quantum state $|\psi\rangle$.

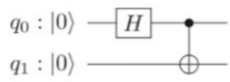
A very special phenomenon takes place when a quantum device has access to more than one qubit: *entanglement*.

One can get an entangled state by applying a Hadamard gate followed by a *CNOT* gate, which operator is given by:

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

The CNOT gate operates on two qubits, a *control* qubit and a *target* qubit, and it works as it: apply the logical NOT operation to the target qubit only if the control qubit has the value 1.

So, if a Hadamard gate is applied in q_0 and then a CNOT whit a control qubit as q_0 and target as q_1, an entanglement state is created.



The result is a superposition of $|00\rangle$ and $|11\rangle$, which cannot be represented as two separated qubits anymore: they are *entangled*. In other words, the value of q_1 is completely connected to the quantum measurement on q_0.

3. IBM Quantum®

IBM provides some useful tools for quantum computing. In this report, IBM Quantum lab was used for implementing all the algorithms, and an introduction for each tool is provided below.

3.1 IBM Quantum Composer®

One can easily graphically build quantum circuits with IBM Quantum Composer. It allows users to dynamically see some useful circuit properties like the outcome probabilities of measurement, statevectors, phase and Q-sphere. In addition, it provides the Qiskit/Python implementation for the currently circuit.

3.2 IBM Quantum Lab®

This toolkit provides a collection of Jupyter Notebooks tutorials created by Qiskit® team. Also, one can creates its own notebook directly in this pane.

4. Travelling Salesman Problem

The Travelling Salesman Problem (TSP) belongs to the class of NP-Hard problems in combinatorial optimization. The problem is:

"Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?"

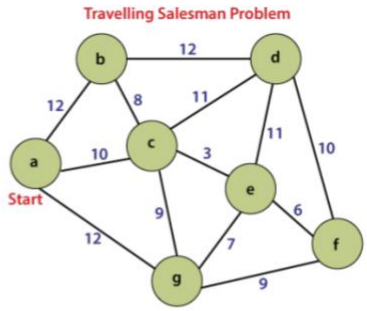


Fig 4. TSP graph representation: The letters represent cities, and the lines values are the distances between them.

The Travelling salesman problem is very important in theoretical computer science. Community has been trying to find an algorithm which can solve it in polynomial time. The reason of its importance is not specifically TSP, but instead the class to which TSP belongs to. It's *the NP class*. If one can find an algorithm for TSP, it will open a wide range of possibilities for the thousands of other problems which belongs to the same class.

The TSP problem is similar to the Hamiltonian cycle problem, as follows:

"The cycle of visiting each vertex once in a graph and returning to the starting vertex is known as a Hamiltonian cycle. Given a graph, determine whether it contains a Hamiltonian cycle or not."

As the Hamiltonian cycle is at the heart of the TSP, these two problems are interconnected and solving one will solve the other.

5. Methodology

To solve TSP, one can consider the whole problem in terms of graphs. The cities are represented as vertices, and the cost/path as edges.

- The problem is approached by encoding the given distances/cost between the cities as phases.
- Each city is connected to other cities with a specific cost associated to each connection.
- Unitary operators are constructed whose eigenvectors are the computational basis states and eigenvalues are various combinations of these phases.
- Then its applied phase estimation algorithm to certain eigenstates which gives all the total distances possible for all the routes.
- After obtaining the distances, a search is made through this information using the quantum search algorithm to find the least possible distance as well the route taken.

This approach provides a quadratic speedup over the classical brute force method for a large number of cities.

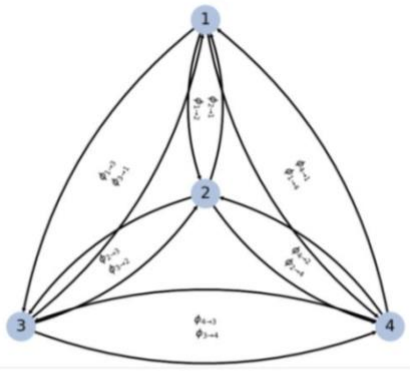


Fig 5. Hamiltonian cycle representation with 4 nodes.

5.1 Phase Estimation

The *quantum phase estimation algorithm* uses *phase kickback* to write the phase of U to the t qubits. When a qubit is used to control the U gate, the qubit will turn proportionally to the phase $e^{2i\pi\theta}$. It's needed to introduce the controlled unitary – CU – that applies the unitary operator U on the target register only if its corresponding control qubit is $|1\rangle$. So, it's needed to decompose these unitaries as *controlled unitaries*. As follows:

$$U_1, U_2, U_3, U_4 \rightarrow CU_1, CU_2, CU_3, CU_4$$

$$U_j = \begin{bmatrix} e^{ia} & 0 & 0 & 0 \\ 0 & e^{ib} & 0 & 0 \\ 0 & 0 & e^{ic} & 0 \\ 0 & 0 & 0 & e^{id} \end{bmatrix}$$

The eigenvalues of this unitary matrix U are estimated using the quantum phase estimation algorithm.

The phases can be normalized to be bound within 0 and 2π once the range of distances between the cities is known.

U is a diagonal matrix since it is a tensor product of n diagonal matrices. This means that the eigenstates of this matrix U are computational basis states with eigenvalues as the corresponding diagonal elements.

The eigenstates are represented in binary form to convert the city to computational basis vectors using a function:

$$|\psi\rangle = \otimes_j |\mathrm{i(j)} - 1\rangle \quad \text{where j} \in [1 \cdots \mathrm{n}]$$

where the function i(j) corresponds to *"from which city the salesman travelled to city j"*?

5.2 Eigenstates

With 4 cities taken, the total combination of all possible Hamiltonian cycle is $\boldsymbol{n = 4! = 24}$. Out of these 24, 6 are *distinct* Hamiltonian cycle.

We are taking the first 6 states, since these states on different circular permutations will give all the states:

Possible States
$1-2-3-4$
$1-2-4-3$
$1-4-2-3$
$1-4-3-2$
$1-3-2-4$
$1-3-4-2$

Tab 1. Possible distinct states

In the binary form, they assume the form:

Sequence path	Eigenstates
$1-2-3-4$	$\|11000110\rangle$
$1-2-4-3$	$\|10000111\rangle$
$1-4-2-3$	$\|10001101\rangle$
$1-4-3-2$	$\|01001110\rangle$
$1-3-2-4$	$\|11001001\rangle$
$1-3-4-2$	$\|01001011\rangle$

Tab 2. Eigenstates in binary form

Considering the same cost from going and returning, there will be 3 sequence paths out of 6. For example, we can take any one of 1–2–3–4 or 1–2–4–3, since cost from 3–4 will be the same as 4–3. So, these 6 will reduce to:

Sequence path	Eigenstates
$1-2-3-4$	$\|11000110\rangle$
$1-4-2-3$	$\|10001101\rangle$
$1-3-2-4$	$\|11001001\rangle$

Tab 3. Eigenstates reduced

6. Results

The phases are normalized to be bound within [0, 2π] once the range of distances between the cities are known.

Phases	Encoded as	Phase value
$\phi_{1\to1}$		0
$\phi_{2\to1}$	a	$\frac{\pi}{2}$
$\phi_{3\to1}$	b	$\frac{\pi}{8}$
$\phi_{4\to1}$	c	$\frac{\pi}{4}$
--	--	--
$\phi_{1\to2}$	d	$\frac{\pi}{2}$
$\phi_{2\to2}$		0
$\phi_{3\to2}$	e	$\frac{\pi}{4}$
$\phi_{4\to2}$	f	$\frac{\pi}{4}$
--	--	--
$\phi_{1\to3}$	g	$\frac{\pi}{8}$
$\phi_{2\to3}$	h	$\frac{\pi}{4}$
$\phi_{3\to3}$		0
$\phi_{4\to3}$	i	$\frac{\pi}{8}$
--	--	--
$\phi_{1\to4}$	j	$\frac{\pi}{4}$
$\phi_{2\to4}$	k	$\frac{\pi}{4}$
$\phi_{3\to4}$	l	$\frac{\pi}{8}$
$\phi_{4\to4}$		0

Tab 4. Phases normalized

The circuit was built and drawn in a Jupyter Notebook.

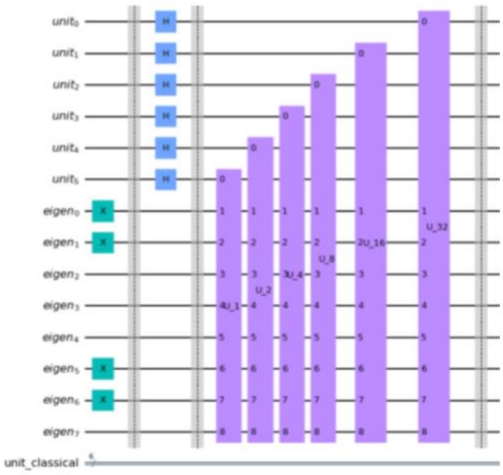


Fig 6. Circuit Implemented - *Part 1*

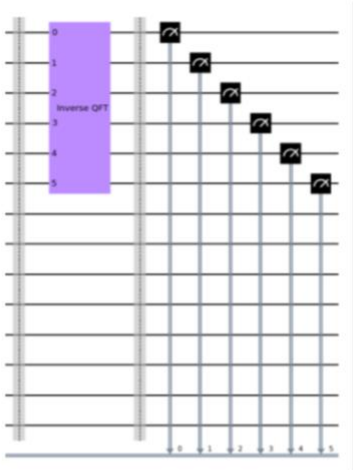


Fig 7. Circuit implemented - *Part 2*

And the result was computed. The number of shots taken was 8129 and in 100% of it, the result was the same: |100100⟩.

```
{'100100': 8192}
```

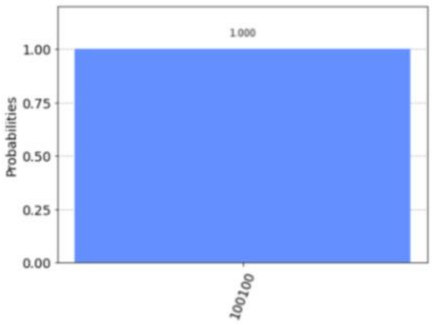


Fig 8. Final measurement.

7. Discussion

This process must be done for all the eigenstates to find the total distance for all the routes. After that, one can use quantum search algorithms to find the minimum of those distances. Thus, the time required scales with the number of eigenstates.

The paper token as reference for this has many circuit errors. They were fixed and the improved version was used.

A good question must be - Does using this process of quantum phase estimation to solve a problem of NP-Hard give a more efficient and optimal algorithm to solve these kinds of problems? No. This process of using phase estimation, which encode the cost as phases is one of the many ways to solve this category of problems. In this process, it was found all the possible Hamiltonian cycle in the graph and based on those cycle, the total cost was calculated. But finding all Hamiltonian cycle of a graph is itself a *NP-Complete* problem. It was used precomputed

Hamiltonian cycle in the beginning to get the eigenstates. And just 4 nodes/cities were used, but for large number of cities, finding all possible Hamiltonian cycle is an extensive work.

8. Conclusion

As mentioned by Srinivasan et al. [4], using the proposed algorithm, it was able to create a database of all possible routes that can be taken along with the distance of each. If one devises a quantum algorithm to find the minimum element in an unsorted array, which is faster than the one it's currently used, then one can use that algorithm to find the minimum. This gives this algorithm flexibility, which then can be exploited in the future to solve the travelling salesman problem much more efficiently. Even though this algorithm deals with a very general case, there are certain cases which cannot be directly solved using it. These are the cases where there are restrictions on routes connecting cities. For instance, city b does not have a route connecting it to city d. This can be thought of as the distance between those cities being infinite. Since this algorithm requires distances that can be normalized such that the total distance for the longest route is less than 2π, this does not bode well for it.

9. References

[1] R.P. Feynman, Simulating physics with computers, Int. J. Theor. Phys. 21 (6) (1982) 467–488.

[2] G. Acampora and A. Vitiello. Implementing evolutionary optimization on actual quantum processors, Int. N. Fisica Nucleare. (2021).

[3] P.W. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, SIAM J. Comput. 26 (5) (1997) 1484–1509.

[4] Karthik Srinivasan, Saipriya Satyajit, Bikash K Behera, and Prasanta K Panigrahi. Efficient quantum algorithm for solving travelling salesman problem: An IBM quantum experience. arXiv:1805.10928v1, 2018.

[5] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John Smolin, Harald Weinfurter. Elementary gates for quantum computation. arXiv:9503016v1, 1995.

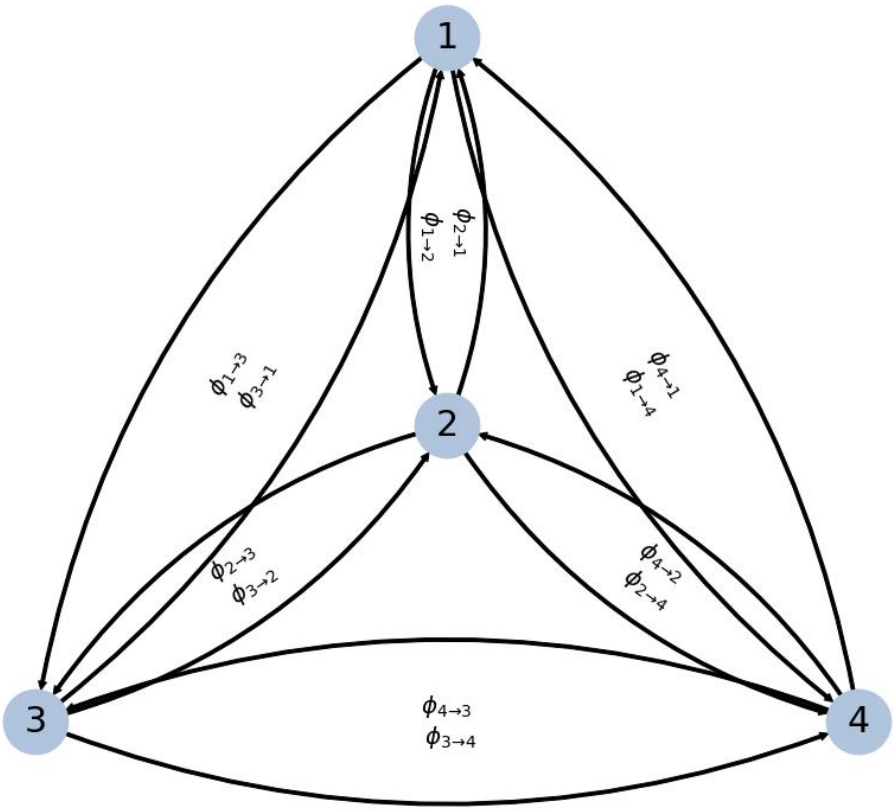
[6] Qiskit Textbook. Solving the Travelling Salesman Problem using Phase Estimation. Avaliable on: https://qiskit.org/. Access date: Apr, 14th 2022.

Apêndice B – Algoritmos Utilizados

Representação do problema utilizando Grafos

```
# Building the graph with nodes = 4

import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph(directed=True)
G.add_node(1)
G.add_node(2)
G.add_node(3)
G.add_node(4)

G.add_edge(1, 2)
G.add_edge(1, 3)
G.add_edge(1, 4)

G.add_edge(2, 1)
G.add_edge(2, 3)
G.add_edge(2, 4)

G.add_edge(3, 1)
G.add_edge(3, 2)
G.add_edge(3, 4)

G.add_edge(4, 1)
G.add_edge(4, 2)
G.add_edge(4, 3)

pos = {1: [0.75, 1.0],
      2: [0.75, 0.15],
      3: [0.5, -0.5],
      4: [1.0, -0.5]}


edge_labels = {(1, 2): '$\\phi_{2\\to 1}$\n $\\phi_{1\\to 2}$',
              (1, 3): '$\\phi_{1\\to 3}$\n $\\phi_{3\\to 1}$',
              (1, 4): '$\\phi_{4\\to 1}$\n $\\phi_{1\\to 4}$',
              (2, 3): '$\\phi_{2\\to 3}$\n $\\phi_{3\\to 2}$',
              (2, 4): '$\\phi_{4\\to 2}$\n $\\phi_{2\\to 4}$',
              (3, 4): '$\\phi_{4\\to 3}$\n $\\phi_{3\\to 4}$'
              }

fig = plt.figure(1, figsize=(14, 10))
nx.draw(G, with_labels=True, node_color='lightsteelblue',
edge_cmap=plt.cm.Blues, pos=pos, connectionstyle='arc3, rad = 0.2',
        node_size=3000, arrowsize=14, arrowstyle='simple', font_size=30)

nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=20,
```

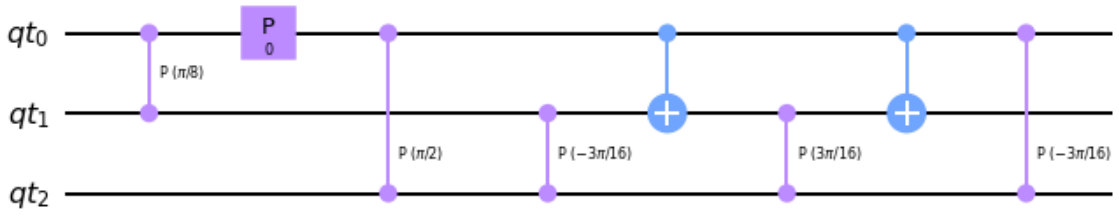
```
bbox=dict(alpha=0))

plt.show()
```



Análise de $CCU1(d-c+a-b)$ utilizando um circuito-teste

```
# Importando as bibliotecas necessárias
from qiskit import QuantumCircuit, Aer, QuantumRegister, ClassicalRegister,
execute,  IBMQ
from qiskit.visualization import plot_histogram, array_to_latex
from qiskit.circuit.library import QFT
from numpy import pi

at = 0
bt = pi/2
ct = pi/8
dt = pi/4

qt = QuantumRegister(3, 'qt')
```

```
qct = QuantumCircuit(qt)

qct.cp(ct - at, qt[0], qt[1])
qct.p(at, qt[0])
qct.cp(bt - at, qt[0], qt[2])

qct.cp((dt - ct + at - bt)/2, qt[1], qt[2])
qct.cx(qt[0], qt[1])
qct.cp(-(dt - ct + at - bt)/2, qt[1], qt[2])
qct.cx(qt[0], qt[1])
qct.cp((dt - ct + at - bt)/2, qt[0], qt[2])

qct.draw()
```



Simulador do Operador Unitário

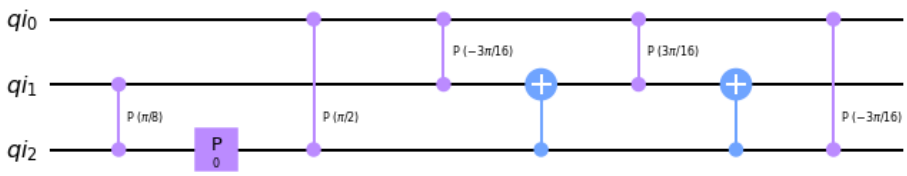
```
backend_unitary_t = Aer.get_backend('unitary_simulator')
job_unitary_t = execute(qct, backend_unitary_t, shots=8192)
count_unitary_t = job_unitary_t.result().get_unitary()
array_to_latex(count_unitary_t, prefix="\\text{Circuit = }\n")
```

$$\text{Circuit} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.92388 + 0.38268i & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & i & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}}(1+i) \end{bmatrix}$$

Versão invertida

```
ai = 0
bi = pi/2
ci = pi/8
```

```
di = pi/4

qi = QuantumRegister(3, 'qi')
qci = QuantumCircuit(qi)


qci.cp(ci - ai, qi[2], qi[1])
qci.p(ai, qi[2])
qci.cp(bi - ai, qi[2], qi[0])

qci.cp((di - ci + ai - bi)/2, qi[1], qi[0])
qci.cx(qi[2], qi[1])
qci.cp(-(di - ci + ai - bi)/2, qi[1], qi[0])
qci.cx(qi[2], qi[1])
qci.cp((di - ci + ai - bi)/2, qi[2], qi[0])

qci.draw()
```



Simulador do Operador Unitário da Versão Invertida

```
backend_unitary_i = Aer.get_backend('unitary_simulator')
job_unitary_i = execute(qci, backend_unitary_i, shots=8192)
count_unitary_i = job_unitary_i.result().get_unitary()
array_to_latex(count_unitary_i, prefix="\\text{Circuit = }\n")
```

$$\text{Circuit} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & i & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.92388 + 0.38268i & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}}(1+i) \end{bmatrix}$$

Construindo o Circuito

```
def controlled_unitary(qc, qubits: list, phases: list): # x,y,z = Qubits;
a,b,c,d = Fases
    qc.cp(phases[2]-phases[0], qubits[0], qubits[1]) # C-U1(c-a)
    qc.p(phases[0], qubits[0]) # U1(a)
    qc.cp(phases[1]-phases[0], qubits[0], qubits[2]) # C-U1(b-a)

    # CCU1(d-c+a-b)
    qc.cp((phases[3]-phases[2]+phases[0]-phases[1])/2, qubits[1], qubits[2])
    qc.cx(qubits[0], qubits[1])
    qc.cp(-(phases[3]-phases[2]+phases[0]-phases[1])/2, qubits[1], qubits[2])
    qc.cx(qubits[0], qubits[1])
    qc.cp((phases[3]-phases[2]+phases[0]-phases[1])/2, qubits[0], qubits[2])
```

$$U = U_1 \otimes U_2 \otimes U_3 \otimes U_4$$

```
def U(times, qc, unit, eigen, phases: list): # a,b,c = fases para U1; d,e,f =
fases para U2; g,h,i = fases para U3; j,k,l = fases para U4; m_list=[m, n, o, p,
q, r, s, t, u, a, b, c, d, e, f, g, h, i, j, k, l]
    controlled_unitary(qc, [unit[0]]+eigen[0:2], [0]+phases[0:3])
    controlled_unitary(qc, [unit[0]]+eigen[2:4], [phases[3]]+[0]+phases[4:6])
    controlled_unitary(qc, [unit[0]]+eigen[4:6], phases[6:8]+[0]+[phases[8]])
    controlled_unitary(qc, [unit[0]]+eigen[6:8], phases[9:12]+[0])
```

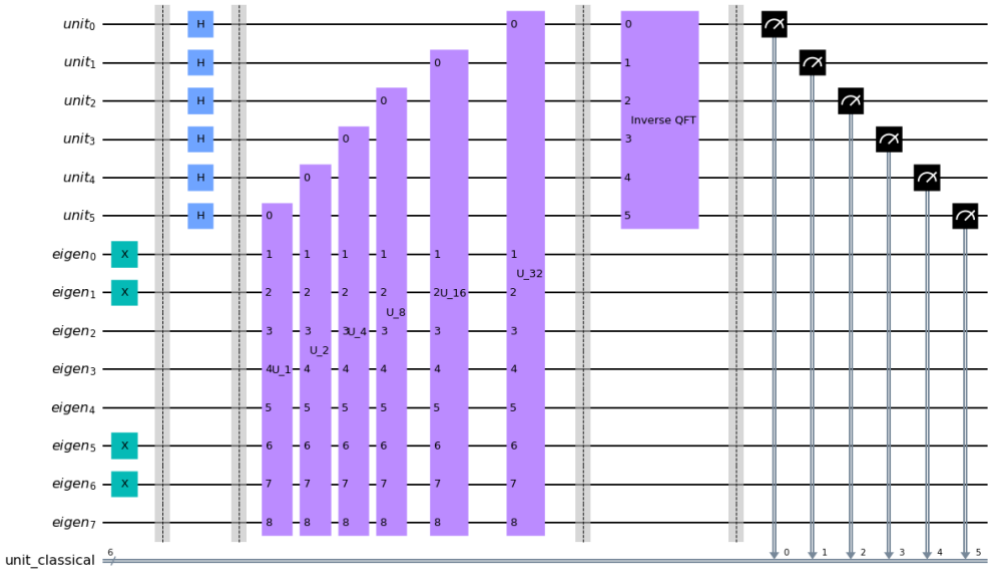
$$CU, CU^2, CU^4, CU^8, CU^{16}, CU^{32}$$

```
def final_U(times, eigen, phases: list):
    unit = QuantumRegister(1, 'unit')
    qc = QuantumCircuit(unit, eigen)
    for _ in range(2**times):
        U(times, qc, unit, eigen, phases)
    return qc.to_gate(label='U'+'_'+(str(2**times)))
```

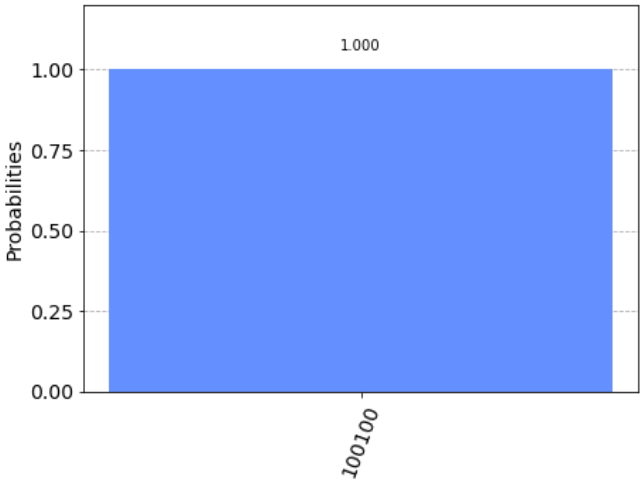
Construindo os autoestados

```
# Guardando os autoestados em um array
eigen_values = ["11000110", "10001101", "11001001"]

# Função para atribuir o gate apropriado do autoestado
def eigenstates(qc, eigen, index):
```

```
    for i in range(0, len(eigen)):
        if eigen_values[index][i] == '1':
            qc.x(eigen[i])
        if eigen_values[index][i] == '0':
            pass
    qc.barrier()
    return qc

Construindo o Circuito

# Inicialização
unit = QuantumRegister(6, 'unit')
eigen = QuantumRegister(8, 'eigen')
unit_classical = ClassicalRegister(6, 'unit_classical')
qc = QuantumCircuit(unit, eigen, unit_classical)
#

# Escolhendo o primeiro autoestado
eigenstates(qc, eigen, 0)

# Aplicando Hadamard nos 'unit' qubits
qc.h(unit[:])
qc.barrier()

# C-U
phases = [pi / 2, pi / 8, pi / 4, pi / 2, pi / 4, pi / 4, pi / 8, pi / 4, pi /
8, pi / 4, pi / 4, pi / 8] # a, b, c, d, e, f, g, h, i, j, k, l
for i in range(0, 6):
    qc.append(final_U(i, eigen, phases), [unit[5-i]] + eigen[:])
#

# Inversa de QFT
qc.barrier()
qft = QFT(num_qubits=len(unit), inverse=True, insert_barriers=True,
do_swaps=False, name='Inverse QFT')
qc.append(qft, qc.qubits[:len(unit)])
qc.barrier()
#

# Medida
qc.measure(unit, unit_classical)
#

# Desenho
qc.draw()
```



Medida

```
backend = Aer.get_backend('qasm_simulator')
job = execute(qc, backend, shots=8192)
count = job.result().get_counts()
print(count)
plot_histogram(count)
```

{'100100': 8192}